RNN Notes — Movie Review Sentiment Classification

1. What is an RNN?

RNN (Recurrent Neural Network) is a neural network designed to work with **sequential data**.

In a movie-review problem, the words have an order:

```
"The movie was not good"
```

The order matters because:

```
"good"
```

and

```
"not good"
```

have different meanings.

A normal Dense layer does not inherently understand this sequence. An RNN maintains information from previous timesteps while processing the current timestep.

Basic idea

```
Word 1 → Word 2 → Word 3 → Word 4 → Word 5
   ↓        ↓        ↓        ↓        ↓
  RNN      RNN      RNN      RNN      RNN
   ↓        ↓        ↓        ↓        ↓
  h1       h2       h3       h4       h5
```

The hidden state carries information from previous words.

---

2. RNN for Movie Review Classification

A typical movie-review model can look like:

```
Movie Review
     ↓
```

```
Tokenization
     ↓
Padding
     ↓
Embedding
     ↓
SimpleRNN
     ↓
Dense(1, sigmoid)
     ↓
Positive / Negative
```

Example:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense

model = Sequential()

model.add(
    Embedding(
        input_dim=vocab_size,
        output_dim=32,
        input_length=100
    )
)

model.add(SimpleRNN(64))

model.add(Dense(1, activation='sigmoid'))
```

Here:

- `vocab_size` = number of unique words/tokens
- `32` = embedding dimension
- `100` = maximum number of words/timesteps
- `64` = number of RNN units
- `Dense(1)` = final binary classification
- `sigmoid` = produces a value between 0 and 1

For example:

```
0.87 → Positive
0.12 → Negative
```

assuming:

```
0 = Negative
1 = Positive
```

---

3. Understanding `input_shape=(timesteps, features)`

RNN input has three dimensions:

```
(batch_size, timesteps, features)
```

But when specifying the input to Keras, we generally provide:

```
input_shape=(timesteps, features)
```

For example:

```
input_shape=(100, 32)
```

means:

```
100 = timesteps
32  = features
```

For a movie review:

```
100 words/tokens
      ↓
Each word represented by
a 32-dimensional embedding
```

So:

```
100 × 32
```

is the input sequence for one review.

If there are 10,000 reviews:

```
(batch_size, 100, 32)

or

(10000, 100, 32)
```

depending on whether the complete dataset or a batch is being represented.

---

4. What Does an RNN Unit Do?

Suppose:

```
SimpleRNN(64)
```

The RNN has **64 hidden units**.

Conceptually:

```
Current word
     +
Previous hidden state
     ↓
    RNN
     ↓
New hidden state
```

Repeated for every timestep:

```
word1 → h1
word2 → h2
word3 → h3
word4 → h4
...
word100 → h100
```

Without `return_sequences=True`, the RNN normally returns only the final hidden state:

```
h100
 ↓
64 values
```

So:

```
SimpleRNN(64)
```

can produce:

```
(batch_size, 64)
```

when used as the final RNN layer.

---

5. What Does a Dense Layer Do?

A Dense layer is a **fully connected layer**.

The easiest way to understand it is:

> **RNN understands the sequence and creates useful features. Dense takes those features and combines them to make a decision.**

Suppose:

```
RNN
 ↓
64 features
```

These 64 values can be thought of as information extracted from the review.

Conceptually:

```
Feature 1 → some learned pattern
Feature 2 → another learned pattern
Feature 3 → another learned pattern
...
Feature 64 → another learned pattern
```

The actual features are learned automatically and are not necessarily individually interpretable.

Dense takes these features and learns how to combine them.

---

6. RNN vs Dense

The most important difference:

RNN = sequence specialist

Dense = feature-combination specialist

RNN	Dense
Designed for sequential data	General-purpose neural-network layer
Processes timesteps	Does not inherently process timesteps sequentially
Maintains hidden state	No recurrent memory
Can use previous timestep information	Treats its input as features
Captures sequence/order relationships	Combines/transforms features
Useful for text, time series, sequences	Useful for classification/regression and feature transformation

For example:

```
Movie Review
     ↓
Embedding
     ↓
RNN
     ↓
"Understand the sequence"
     ↓
Dense
     ↓
"Combine the learned features"
     ↓
Prediction
```

---

7. RNN + Dense(1)

The simplest sentiment model is:

```
model = Sequential()

model.add(Embedding(vocab_size, 32, input_length=100))

model.add(SimpleRNN(64))
```

```
model.add(Dense(1, activation='sigmoid'))
```

Architecture:

```
Embedding
   ↓
(100, 32)
   ↓
SimpleRNN(64)
   ↓
(64)
   ↓
Dense(1)
   ↓
0.87
```

This is often enough for a simple sentiment-classification problem.

---

8. What Happens if We Add Dense(64)?

Now consider:

```
model = Sequential()

model.add(Embedding(vocab_size, 32, input_length=100))

model.add(SimpleRNN(64))

model.add(Dense(64, activation='relu'))

model.add(Dense(1, activation='sigmoid'))
```

The flow becomes:

```
Embedding
    ↓
SimpleRNN(64)
    ↓
64 features
    ↓
Dense(64, ReLU)
    ↓
64 transformed features
    ↓
```

```
Dense(1, sigmoid)
    ↓
Prediction
```

The Dense(64) layer does **not** process the review word-by-word.

Instead, it takes the representation created by the RNN and learns additional combinations of those features.

---

9. Why Add Dense After RNN?

An additional Dense layer gives the model **more learning capacity**.

Without it:

```
RNN → Dense(1)
```

The RNN representation goes almost directly into the final classifier.

With it:

```
RNN → Dense(64) → Dense(1)
```

the model gets an additional stage in which it can transform and combine the RNN's learned features.

This can be useful when the relationship between the RNN features and the final output is complex.

---

10. Disadvantages of Adding Dense After RNN

Adding Dense is **not always better**.

A. Overfitting

More layers and parameters give the model more capacity.

For example:

```
Without Dense:

Training accuracy   = 90%
Validation accuracy = 87%
```

But after adding Dense:

```
Training accuracy   = 98%
Validation accuracy = 80%
```

This indicates that the larger model may be **overfitting**.

The model performs better on training data but worse on unseen data.

B. More Parameters

Suppose the RNN outputs 64 values:

```
SimpleRNN(64)
```

and we add:

```
Dense(64)
```

The Dense layer has approximately:

```
64 × 64 + 64
= 4160 parameters
```

The additional parameters increase:

- computation
- memory usage
- training complexity
- potential for overfitting

C. It May Not Add Useful Information

The RNN may already have created a sufficiently useful representation.

Therefore:

```
RNN → Dense(1)
```

could perform better than:

```
RNN → Dense(64) → Dense(1)
```

There is no rule saying:

> More layers = better accuracy.

The correct approach is to compare validation performance.

---

11. Important Lesson About Dense

Do **not** add Dense layers simply because:

> "More layers should make the model better."

Instead:

```
Start with a simple model
        ↓
Evaluate validation performance
        ↓
Check underfitting/overfitting
        ↓
Increase capacity only if necessary
```

A simpler model that generalizes better is preferable to a more complicated model.

---

12. What Happens if We Add Another RNN?

Suppose we start with:

```
Embedding
    ↓
SimpleRNN(64)
    ↓
Dense(1)
```

Now we add another RNN:

```
Embedding
    ↓
SimpleRNN(64)
    ↓
```

```
SimpleRNN(32)
    ↓
Dense(1)
```

This is called a:

Stacked RNN

Multiple RNN layers are stacked on top of each other.

---

13. Why Do We Need `return_sequences=True`?

This is one of the most important concepts in stacked RNNs.

Consider:

```
model.add(SimpleRNN(64))
model.add(SimpleRNN(32))
```

The first RNN normally returns only its final hidden state:

```
100 timesteps
     ↓
SimpleRNN(64)
     ↓
64 values
```

But the second RNN expects a **sequence**:

```
timestep 1 → representation
timestep 2 → representation
timestep 3 → representation
...
timestep 100 → representation
```

Therefore the first RNN must return the output from every timestep.

We use:

```
return_sequences=True
```

---

14. `return_sequences=True`

Without:

```
SimpleRNN(64)
```

conceptually:

```
Input
(100, 32)
   ↓
RNN
   ↓
(64)
```

Only the final representation is returned.

With:

```
SimpleRNN(64, return_sequences=True)
```

we get:

```
Input
(100, 32)
   ↓
RNN
   ↓
(100, 64)
```

The sequence dimension is preserved.

---

15. Two-Layer Stacked RNN

Correct implementation:

```
model = Sequential()

model.add(
    Embedding(
        vocab_size,
        32,
```

```
        input_length=100
    )
)

model.add(
    SimpleRNN(
        64,
        return_sequences=True
    )
)

model.add(
    SimpleRNN(32)
)

model.add(
    Dense(1, activation='sigmoid')
)
```

Shape flow:

```
Embedding
(100, 32)
     ↓
RNN(64, return_sequences=True)
(100, 64)
     ↓
RNN(32)
(32)
     ↓
Dense(1)
(1)
```

---

16. Three-Layer Stacked RNN

We can add another RNN:

```
model = Sequential()

model.add(Embedding(vocab_size, 32, input_length=100))

model.add(
    SimpleRNN(
        128,
        return_sequences=True
    )
```

```
)

model.add(
    SimpleRNN(
        64,
        return_sequences=True
    )
)

model.add(
    SimpleRNN(32)
)

model.add(Dense(1, activation='sigmoid'))
```

Shape flow:

```
Embedding
    ↓
100 × 32
    ↓
RNN 128
    ↓
100 × 128
    ↓
RNN 64
    ↓
100 × 64
    ↓
RNN 32
    ↓
32
    ↓
Dense(1)
    ↓
1
```

---

17. Rule for `return_sequences`

For stacked RNNs:

```
SimpleRNN(128, return_sequences=True)
SimpleRNN(64,  return_sequences=True)
SimpleRNN(32)
```

The rule is:

Every RNN layer before the final RNN generally needs `return_sequences=True`.

The final RNN does not need it when we want one final prediction for the entire movie review.

Think:

```
RNN 1 → sequence
   ↓
RNN 2 → sequence
   ↓
RNN 3 → final representation
   ↓
Dense → prediction
```

---

18. Why Use a Stacked RNN?

A single RNN performs sequence processing at one level:

```
Embedding
    ↓
RNN
    ↓
Dense
```

A stacked RNN provides multiple levels of sequence processing:

```
Embedding
    ↓
RNN Layer 1
    ↓
RNN Layer 2
    ↓
RNN Layer 3
    ↓
Dense
```

Conceptually:

```
RNN Layer 1
↓
learns basic sequential patterns

RNN Layer 2
```

```
↓
learns higher-level sequential patterns

RNN Layer 3
↓
learns even more complex representations
```

These descriptions are conceptual; the network automatically learns the actual representations.

---

19. Stacked RNN vs RNN + Dense

This distinction is very important.

RNN + Dense

```
Embedding
    ↓
RNN
    ↓
Dense
    ↓
Dense
```

The additional Dense layer performs **feature transformation**.

It does not add another recurrent/sequence-processing stage.

Stacked RNN

```
Embedding
    ↓
RNN
    ↓
RNN
    ↓
Dense
```

The second RNN performs **another level of sequence processing**.

So:

```
RNN → RNN
```

means:

"Process the sequence again."

Whereas:

```
RNN → Dense
```

means:

"Take the representation created by the RNN and transform/combine its features."

---

20. Advantages of Stacked RNN

Advantage 1 — More representation capacity

Multiple RNN layers can learn more complex representations.

```
RNN 1
 ↓
basic patterns

RNN 2
 ↓
higher-level patterns
```

Advantage 2 — Better for complex sequential relationships

For sufficiently complex sequence problems, a deeper recurrent architecture can capture patterns that a single layer may not represent as effectively.

Advantage 3 — Hierarchical processing

Each RNN layer receives the sequence representation from the layer below it.

```
Input sequence
      ↓
RNN 1
      ↓
sequence representation
      ↓
RNN 2
      ↓
higher-level representation
```

---

21. Disadvantages of Stacked RNN

Disadvantage 1 — More parameters

More RNN layers mean more parameters.

```
More layers
    ↓
More parameters
    ↓
More computation
```

Disadvantage 2 — More risk of overfitting

If your dataset is small:

```
Complex model
     ↓
Can memorize training data
     ↓
Poorer validation performance
```

Disadvantage 3 — Training becomes harder

RNNs already have difficulties with long-term dependencies and gradients.

Stacking more recurrent layers can make training more difficult.

Disadvantage 4 — More computationally expensive

A stacked RNN takes longer to train and uses more memory than a single RNN.

---

22. Important Comparison

Architecture	What it adds	Main purpose
`RNN → Dense`	Basic classifier	Sequence → prediction
`RNN → Dense → Dense`	More feature processing	More nonlinear feature combinations
`RNN → RNN → Dense`	More sequence processing	Deeper sequential representation
`RNN → RNN → RNN → Dense`	Even deeper sequence processing	More complex sequential representation

---

23. Simple Mental Model

Remember these three roles:

```
RNN
↓
"Understand the sequence."

Dense(64)
↓
"Combine and transform the features."

Dense(1)
↓
"Make the final prediction."
```

For stacked RNN:

```
RNN 1
↓
"Process the sequence."

RNN 2
↓
"Process the sequence representation at a deeper level."

Dense
↓
"Make the prediction."
```

---

24. Recommended Starting Point for Movie Reviews

Don't immediately build a very deep model.

Start with:

```
model = Sequential()

model.add(
    Embedding(
        vocab_size,
        32,
```

```
        input_length=100
    )
)

model.add(SimpleRNN(64))

model.add(Dense(1, activation='sigmoid'))
```

Then experiment with:

Experiment 1 — Add Dense

```
Embedding
    ↓
SimpleRNN(64)
    ↓
Dense(64, relu)
    ↓
Dense(1, sigmoid)
```

Experiment 2 — Stack RNN

```
Embedding
    ↓
SimpleRNN(64, return_sequences=True)
    ↓
SimpleRNN(32)
    ↓
Dense(1, sigmoid)
```

Then compare **validation accuracy and validation loss**.

Do not assume either architecture will automatically be better.

---

25. Final Takeaway

The most important concepts from this discussion are:

```
1. RNN understands sequential information.

2. Dense does not inherently understand sequence/order.

3. RNN produces a representation of the sequence.

4. Dense can combine and transform that representation.
```

```
5. Adding Dense can improve performance, but can also cause
   overfitting or simply fail to improve the model.

6. Multiple RNN layers are called a Stacked RNN.

7. When stacking RNNs, intermediate RNN layers generally use:

   return_sequences=True

8. The final RNN can return only its final representation when
   doing whole-review classification.

9. More layers do NOT automatically mean better accuracy.

10. Always compare validation/test performance, not just
    training accuracy.
```

One-line memory trick

```
RNN  → sequence understanding
Dense → feature combination
Stacked RNN → deeper sequence understanding
```